Julio Eduardo Gustavo Alvarado Ramirez 202307764

# Ensayo LLVM

Durante el desarrollo de la tarea de OLC1, tuvimos la oportunidad de poner en práctica los conocimientos adquiridos en clase mediante la creación de un compilador utilizando C, Flex, Bison y LLVM. El objetivo principal fue desarrollar un programa capaz de analizar código fuente, verificar que fuera correcto y generar instrucciones que pudieran ejecutarse en una arquitectura específica.

Para comenzar, utilizamos Flex para realizar el análisis léxico, cuya función fue reconocer los diferentes elementos del lenguaje, como palabras reservadas, identificadores, números y operadores. Luego, utilizamos Bison para el análisis sintáctico, que nos permitió comprobar que las instrucciones respetaran la gramática definida. A partir de este proceso construimos un árbol de sintaxis abstracta (AST), el cual representaba de manera organizada las instrucciones del programa y facilitaba su procesamiento.

Después implementamos el análisis semántico, utilizando una tabla de símbolos para almacenar información sobre las variables declaradas. En esta etapa verificamos aspectos como la existencia de las variables y la compatibilidad de tipos en las operaciones. Esto fue importante porque comprendimos que un programa puede estar correctamente escrito en cuanto a su sintaxis, pero aun así contener errores semánticos.

Una de las partes más importantes de la tarea fue utilizar LLVM para generar código intermedio (LLVM IR). A partir del árbol sintáctico generamos instrucciones para operaciones aritméticas, manejo de variables e impresión de resultados. La importancia de LLVM fue que nos permitió trabajar con una representación intermedia que posteriormente podía transformarse en código para diferentes arquitecturas. En nuestro caso, utilizamos LLVM para generar código ARM compatible con el procesador Cortex-M0+ de la Raspberry Pi Pico. También logramos integrar una instrucción para controlar el LED de la placa y generar archivos de firmware en formato UF2.

Durante el desarrollo encontré diferentes dificultades relacionadas con la generación de código, las herramientas de compilación y la integración con el hardware. Sin embargo, mediante pruebas y correcciones conseguimos que las 117 pruebas automatizadas del compilador fueran satisfactorias y generar el firmware correspondiente, quedando pendiente comprobar su funcionamiento directamente en la placa.

En conclusión, esta tarea nos permitió comprender mejor cómo funciona un compilador desde que recibe el código fuente hasta que genera instrucciones para una máquina. Aprendimos la importancia de cada una de sus etapas, cómo se relacionan entre sí y cómo herramientas como Flex, Bison y LLVM facilitan su implementación. Además, comprendimos que desarrollar un compilador requiere organización, pruebas constantes y una buena integración entre sus componentes.